

Introduction

Modern malware uses packing and encryption to evade signature-based antivirus. MalwareGuard is an AI-driven defensive tool that classifies Windows executables as benign or malicious from the structural metadata in their PE (Portable Executable) headers — signals attackers cannot hide without breaking the file.

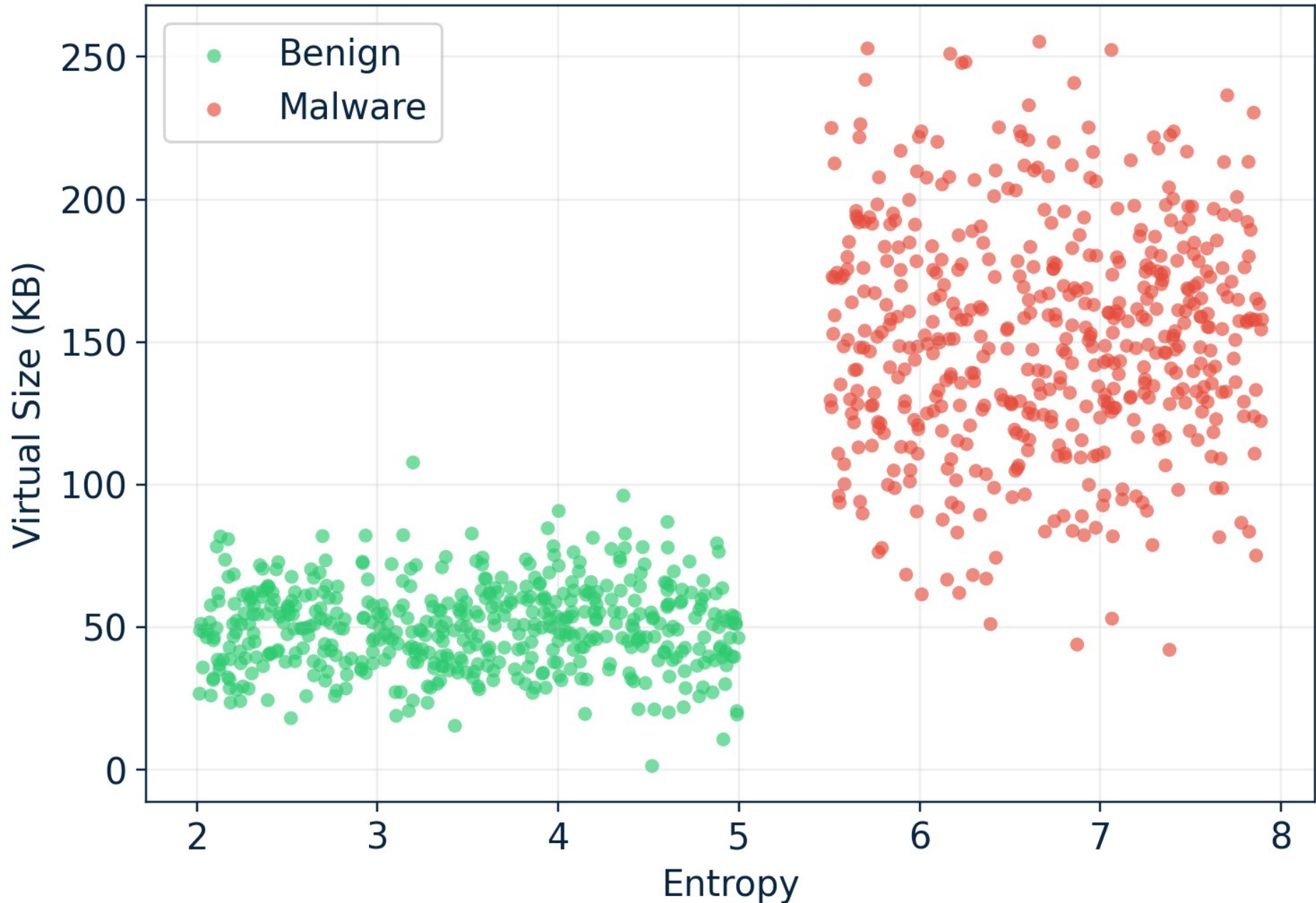
Goals:

- Build an ML classifier from PE-header features
- Test its robustness to adversarial evasion
- Deliver an analyst MVP with logging & alerting

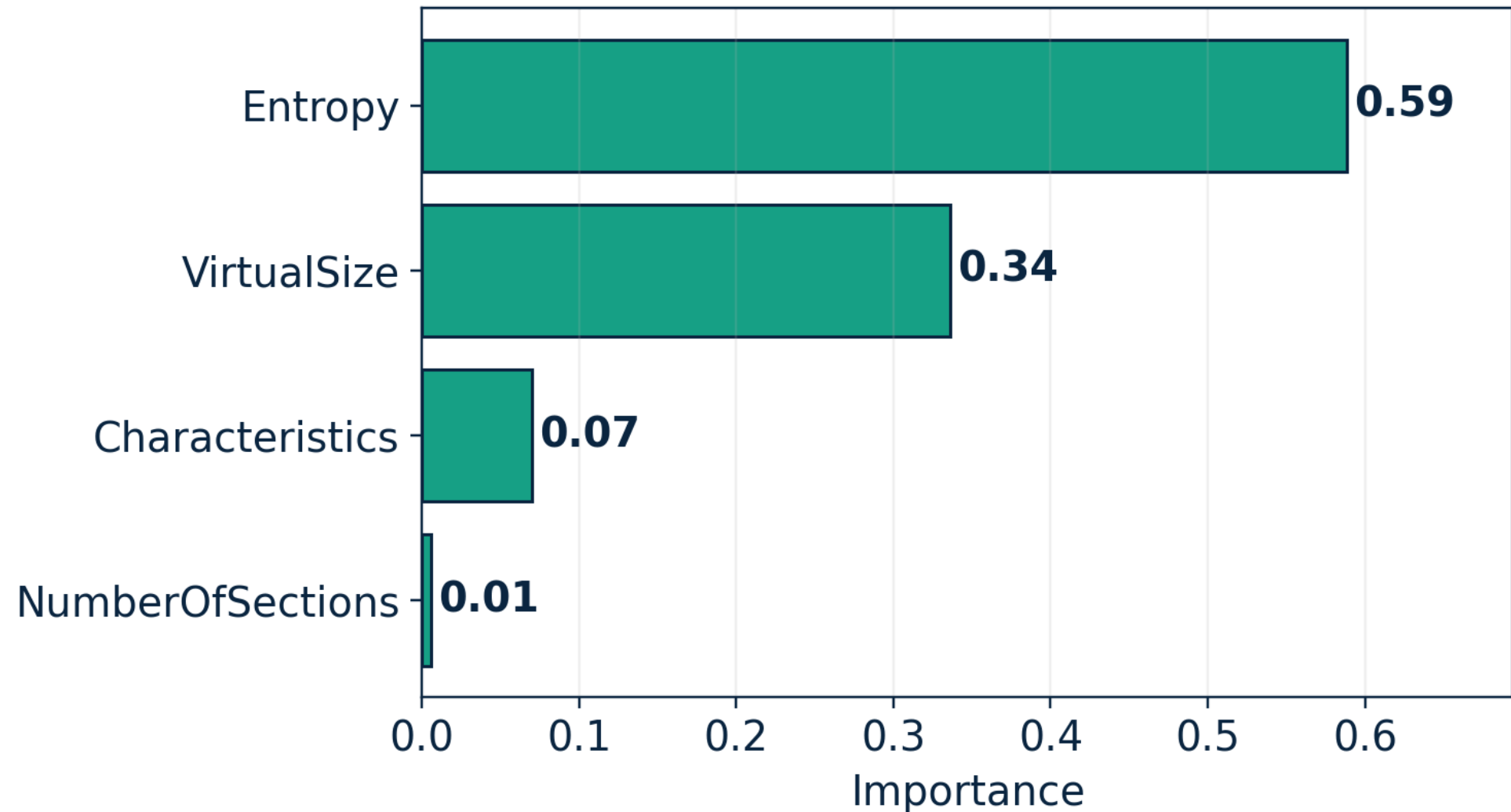
Methodology & System

- Data: 1,000 labeled samples — VirtualSize, Entropy, NumberOfSections, Characteristics (80/20 split)
- Model: Random Forest (100 trees); baseline = threshold heuristic
- Stack: Python · scikit-learn · Streamlit · Joblib
- Pipeline: features → inference → prediction + confidence → audit log + visual alert

Structural Separation of Benign vs. Malicious PE Files

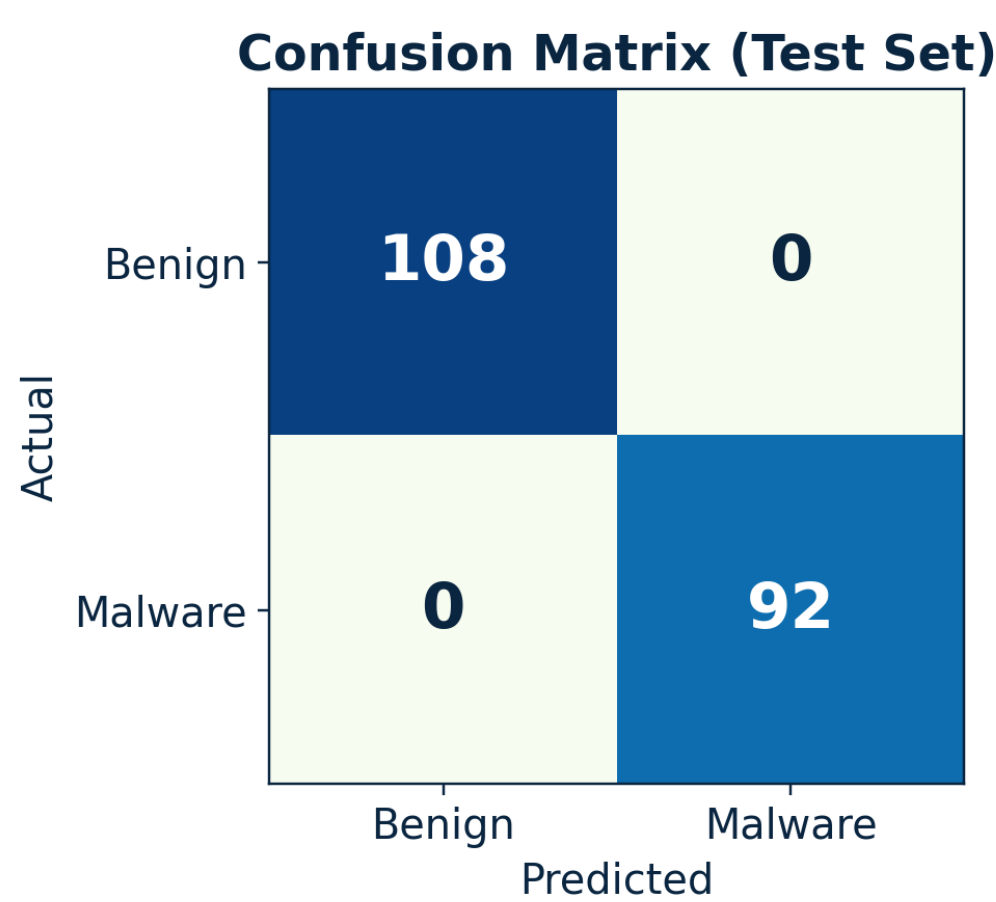


Random Forest Feature Importance

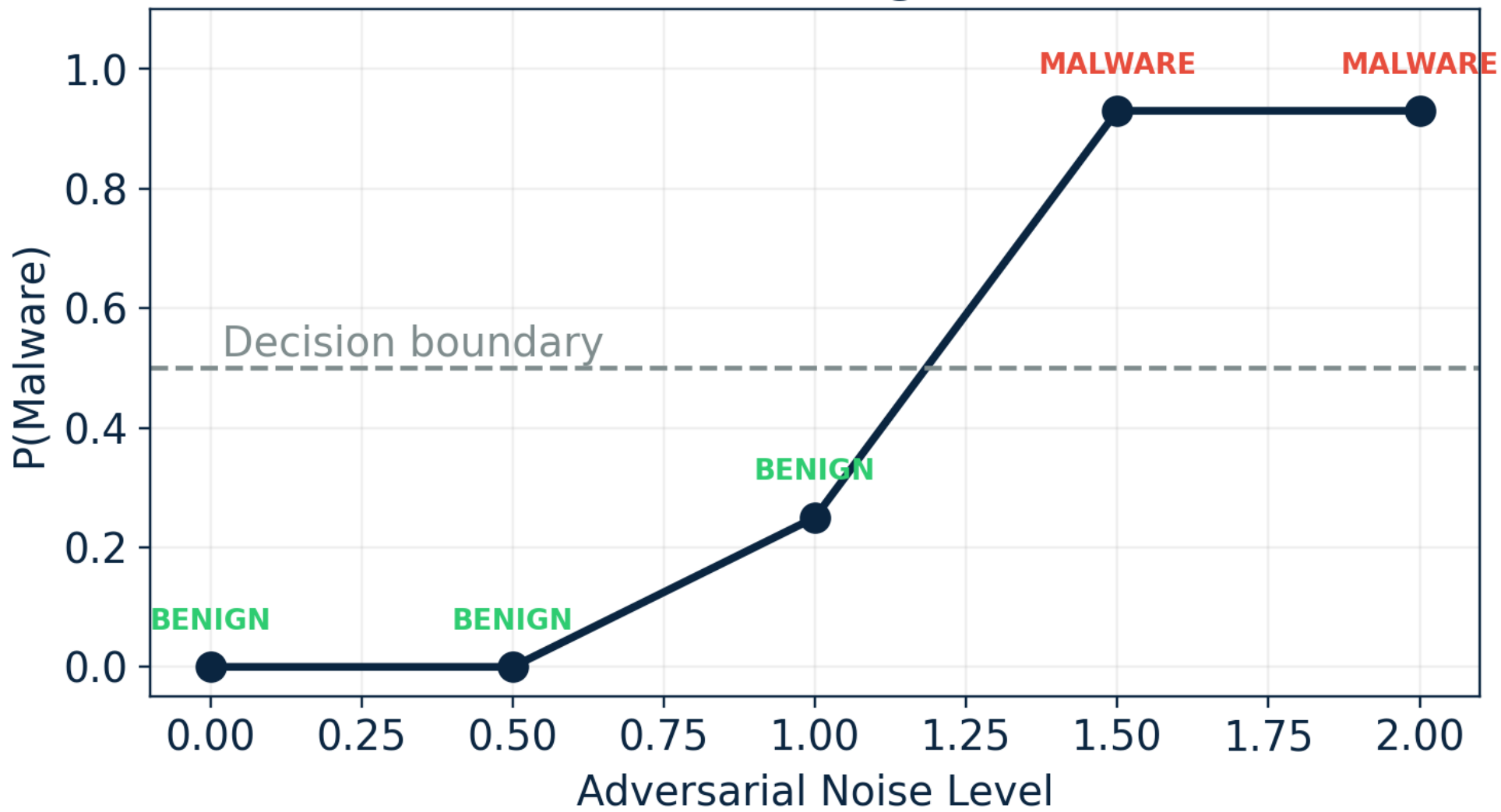


Results & Evaluation

- 100% accuracy / F1 = 1.0 on the held-out test set
- Entropy & VirtualSize are the most discriminative features
- Adversarial test: raising Entropy + VirtualSize flips a benign file to “Malware” exactly as it enters the malicious range — the model reacts to the intended security signals



Adversarial Robustness: Benign → Malware Transition



Conclusions

- All three goals achieved: working classifier, robustness evaluation, deployable MVP
- Structural PE-header analysis is an effective, lightweight complement to signature-based detection
- Results meet the targets defined at project start

Discussions

Future work:

- Retrain on real corpora (EMBER, MalwareBazaar)
- Add deep models (CNN/LSTM) over API-call / byte sequences
- Harden against entropy-balancing evasion; integrate into a SOC triage pipeline



Source code & video ▶